

Chapter 3 <word2vec>

추론 기반 기법 : word2vec

3.1. 추론 기반 기법과 신경망

단어를 벡터로 표현하는 방법중 성공적인 기법은 2가지 이다.

1. 통계 기반 기법
2. 추론 기반 기법

이 두개 모두 단어의 의미를 얻는 방식은 다르지만, "분포 가설"을 이용한다는 점은 동일하다.

통계 기반 기법의 문제를 지적하고 대안인 추론 기반 기법의 이점을 거시적 관점에서 설명하고자 한다.

추가적으로 word2vec의 전처리를 위해 신경망을 이용한 "단어" 처리 예시를 보여주겠다.

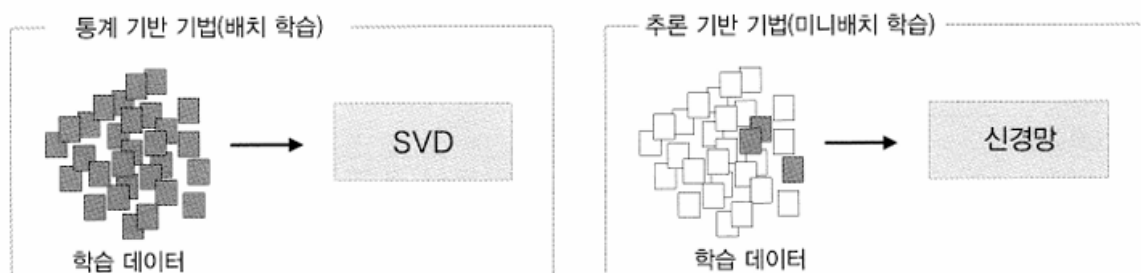
3.1.1. 통계 기반 기법의 문제

단어의 빈도를 기초로 단어를 표현

단어의 동시발생 행렬을 만들고 SVD를 행렬에 적용하여 밀집벡터 (= 단어의 분산표현)

하지만 이렇게 되면 대규모의 말뭉치를 다룰 때 문제가 생긴다. (시간 소요 관련)

그림 3-1 통계 기반 기법과 추론 기반 기법 비교



추론 기반 기법:

보통 미니배치를 이용한 학습을 한다

미니배치 학습이란? 신경망이 한 번에 소량(미니배치)의 학습 샘플씩 반복해서 가중치 갱신

↔ 통계 기반 기법 :

보통 배치를 이용한 학습을 한다

말뭉치 전체의 통계를 이용해 1회의 처리만에 단어의 분산 표현을 얻음

3.1.2. 추론 기반 기법 개요

맥락이 주어졌을 때 빈칸을 추측하는 작업

그림 3-2 주변 단어들을 맥락으로 사용해 "?"에 들어갈 단어를 추측한다.

you ? goodbye and I say hello.

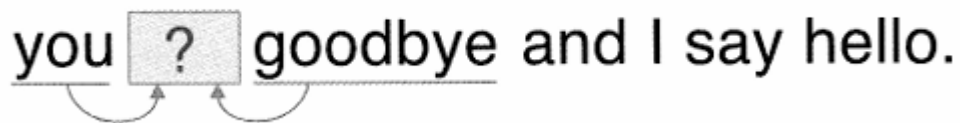
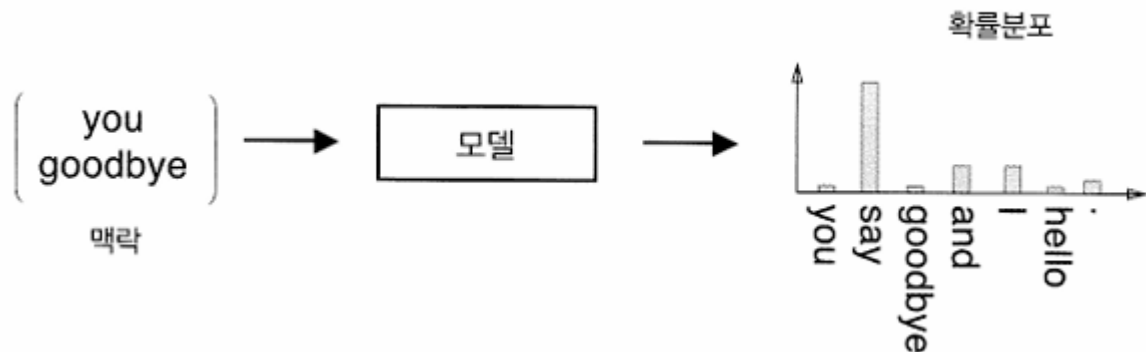


그림 3-3 추론 기반 기법: 맥락을 입력하면 모델은 각 단어의 출현 확률을 출력한다.



3.1.3. 신경망에서의 단어 처리

단어를 "고정 길이의 벡터"로 변환해야하는데 이때 사용하는 대표적인 방법은

★ 단어를 원 핫 표현으로 변환하는 것 ★

그림 3-4 단어, 단어 ID, 원핫 표현

단어(텍스트)	단어 ID	원핫 표현
$\begin{pmatrix} \text{you} \\ \text{goodbye} \end{pmatrix}$	$\begin{pmatrix} 0 \\ 2 \end{pmatrix}$	$\begin{pmatrix} (1, 0, 0, 0, 0, 0, 0) \\ (0, 0, 1, 0, 0, 0, 0) \end{pmatrix}$

이런식으로 각 단어별로 인덱스를 부여하는 느낌

⇒ 뉴런의 수를 고정 가능

그림 3-5 입력층의 뉴런: 각 뉴런이 각 단어에 대응(해당 뉴런이 1이면 검은색, 0이면 흰색)

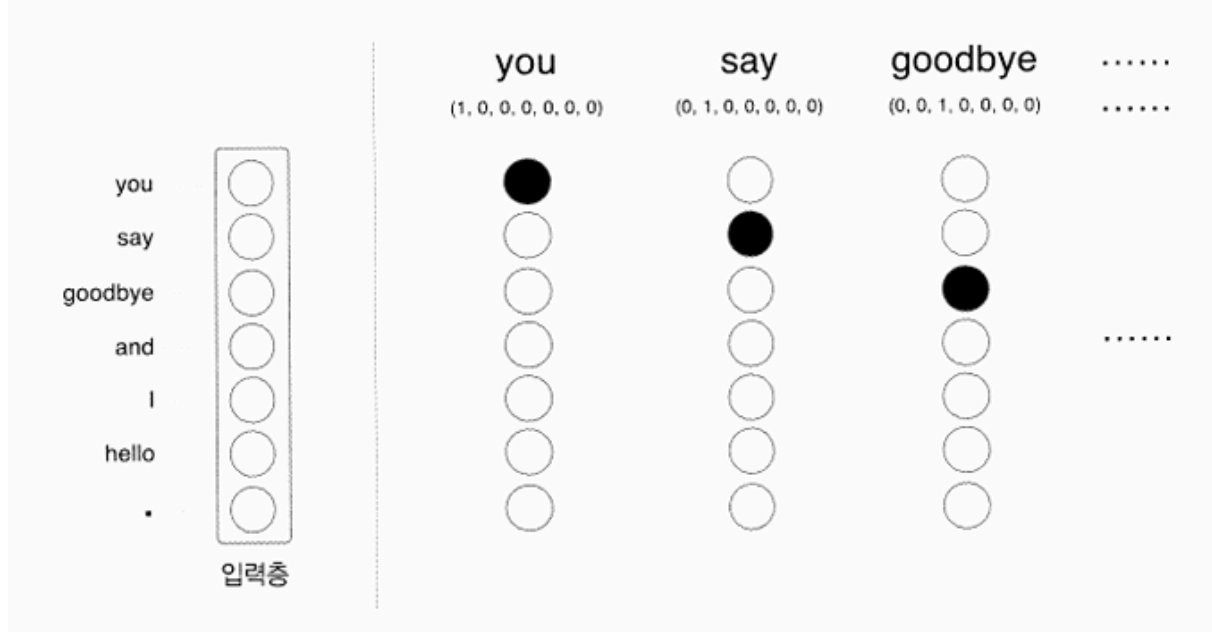
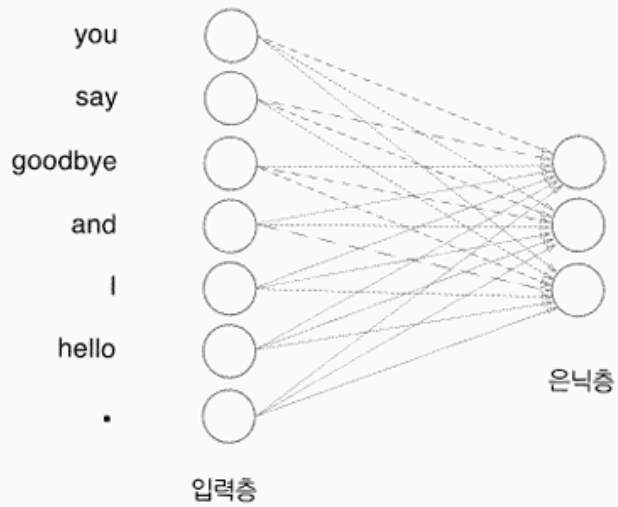
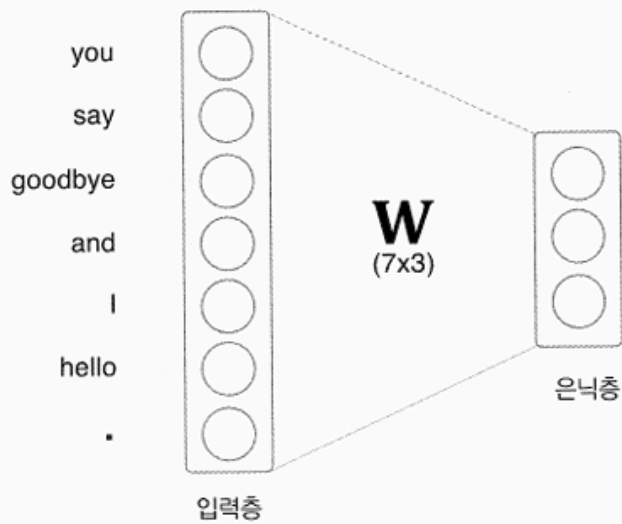


그림 3-6 완전연결계층에 의한 변환: 입력층의 각 뉴런은 7개의 단어 각각에 대응(은닉층 뉴런은 3개를 준비함)



이렇게 각 단어별로 하나의 뉴런에 대응되게 한 후 가중치를 담아서 은닉층으로 보냄
 ⇒ 입력층 뉴런과 가중합이 은닉층 뉴런이 된다.

그림 3-7 완전연결계층에 의한 변환을 단순화한 그림(완전연결계층의 가중치를 7×3 크기의 **W**라는 행렬로 표현)



```
import numpy as np
```

```
c = np.array([[1,0,0,0,0,0,0]])  
w = np.random.randn(7,3)  
h = np.matmul(c,w)
```

```
print(h)
```

```
[[ -0.73012465  1.00429245  1.66225419]]
```

3.2. 단순한 word2vec

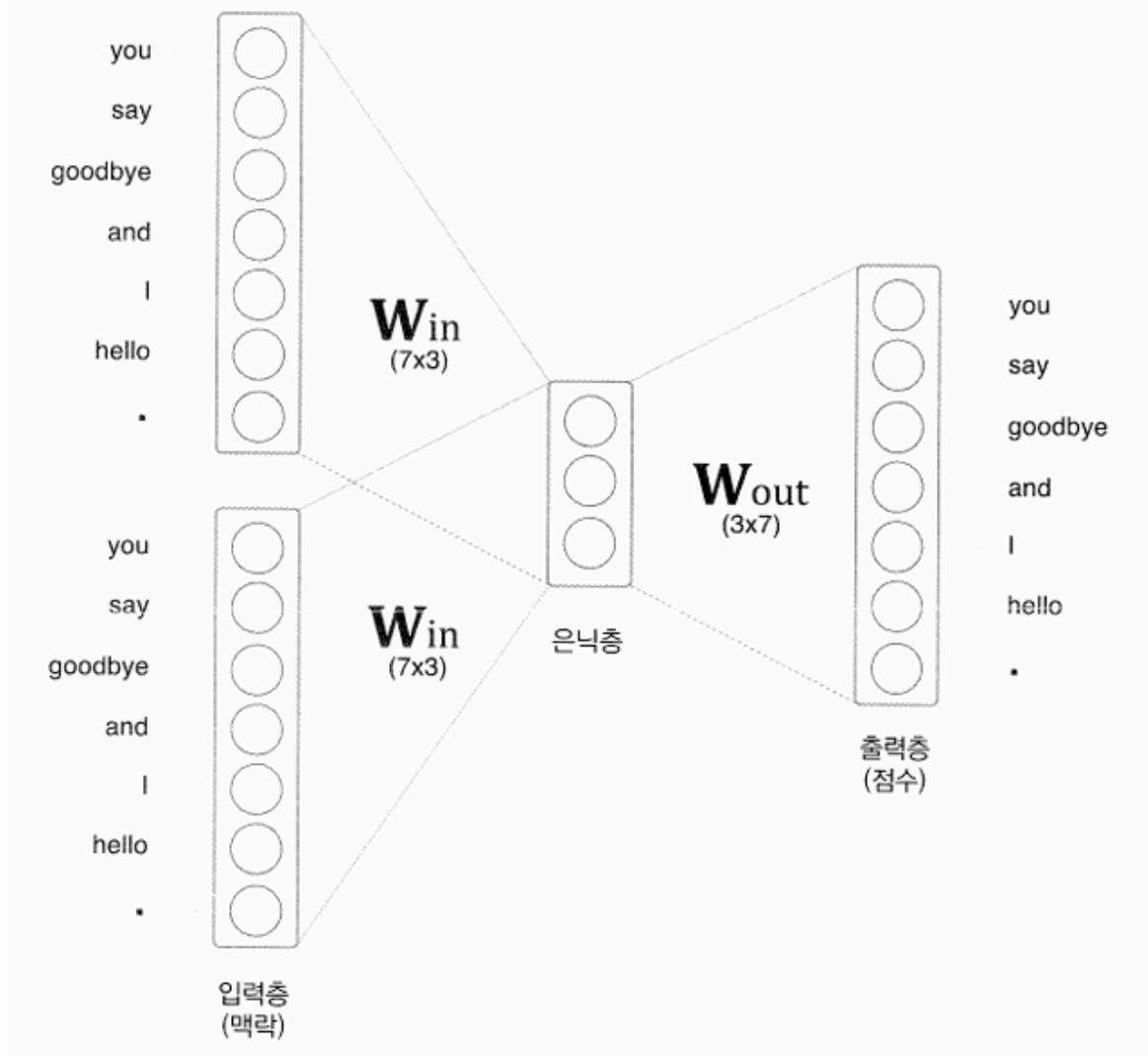
이번 절에서 사용할 신경망은 **CBOW (continuous bag-of-words)** 이다.

3.2.1. CBOW 모델의 추론 처리

맥락으로부터 target을 추측하는 용도의 신경망이다 (참고로 주변 단어들은 "맥락")

CBOW 모델의 입력은 맥락으로 위에서 나왔던 것중 "you", "goodbye" 같은 단어들이다.

그림 3-9 CBOW 모델의 신경망 구조



** 입력층이 2개인 이유

→ 맥락으로써 고려할 단어를 2개로 정했기 때문

→ 맥락으로 포함시킬 단어가 n개면 n개의 입력층이 필요하다

은닉층의 뉴런은 입력층의 완전 연결 계층에 의해 변환된 값이 된다.

so, 입력층이 여러개여도 전체를 "평균" 내면 된다는 뜻.

첫번째 입력층을 h_1 , 두번째 입력층을 h_2 라고 하면, 은닉층의 뉴런은 $1/2(h_1+h_2)$ 가 된다.

출력층은 각 단어의 점수를 뜻하고 값이 높을수록 대응하는 단어일 확률이 높아지는 것

→ softmax를 활용하면 "확률"을 얻을 수 있다.

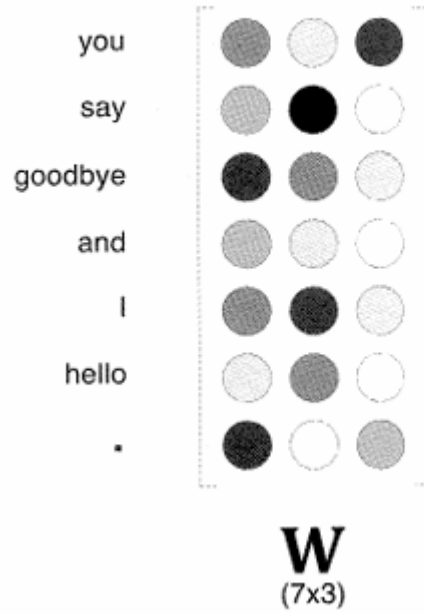
가중치 행렬 W_{in} (7×3)

- 이 행렬은 단어 임베딩을 위한 행렬
- one-hot 벡터(7×1)를 곱하면 은닉층 벡터(3차원)**가 나옴.
 - → 즉, $W_{in} (7 \times 3) \times x (7 \times 1) = h (3 \times 1)$
- 여기서 3은 **embedding dimension (단어 벡터의 차원 수)**이고,
 - 7은 **단어의 개수 (vocabulary size)**

즉, W_{in} 의 크기가 7×3인 이유는

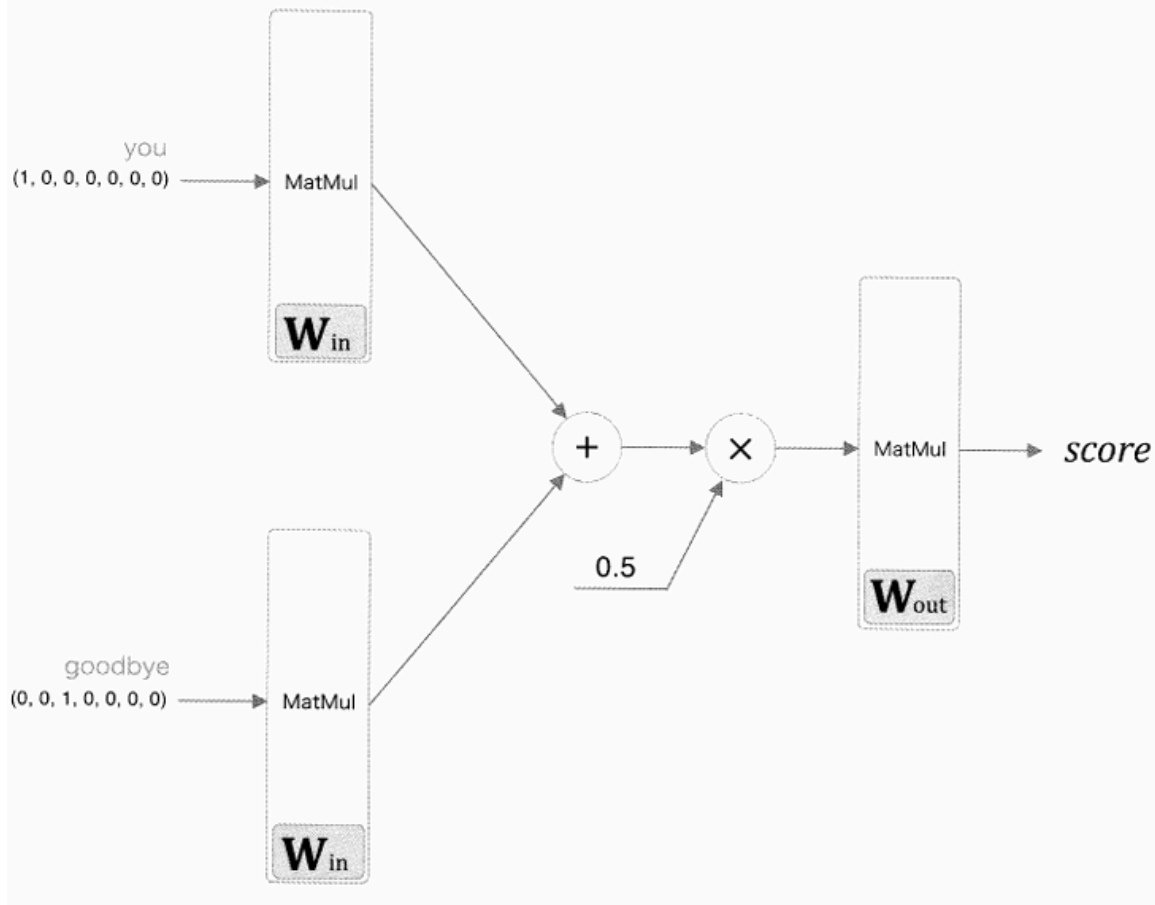
- 7개의 단어 각각을
- 3차원 임베딩 벡터로 바꾸기 위해서
-
- one-hot 벡터 자체는 7×1
- 하지만 W_{in} 은 단어를 임베딩 벡터로 바꾸는 역할을 하므로 7×3
- 각 단어마다 임베딩 차원이 3이기 때문 (그냥 임의로 3 정함)

그림 3-10 기중치의 각 행이 해당 단어의 분산 표현이다.



여기까지 뉴런 관점, 이후로는 계층 관점

그림 3-11 계층 관점에서 본 CBOW 모델의 신경망 구성: MatMul 계층에서 사용하는 가중치($\mathbf{W}_{in}$, $\mathbf{W}_{out}$)는 해당 계층 안으로 넣었음

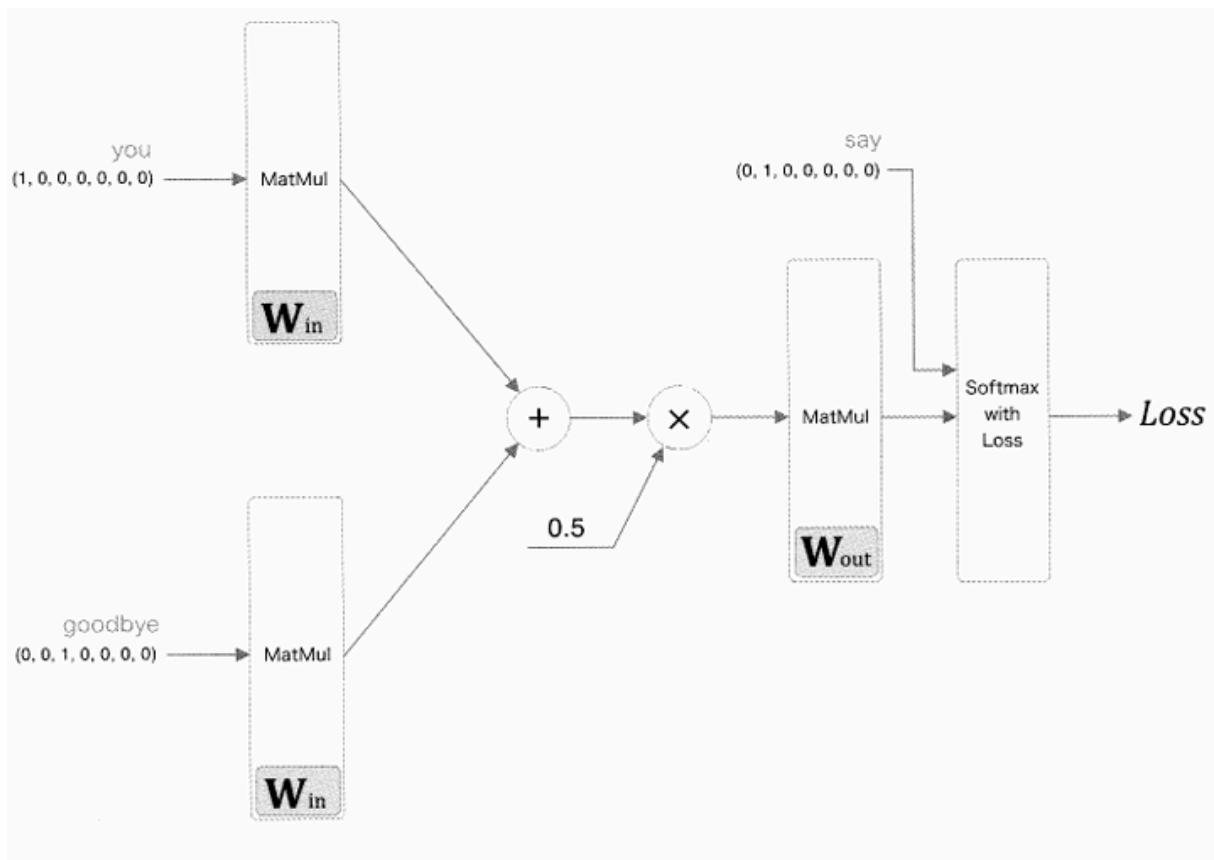
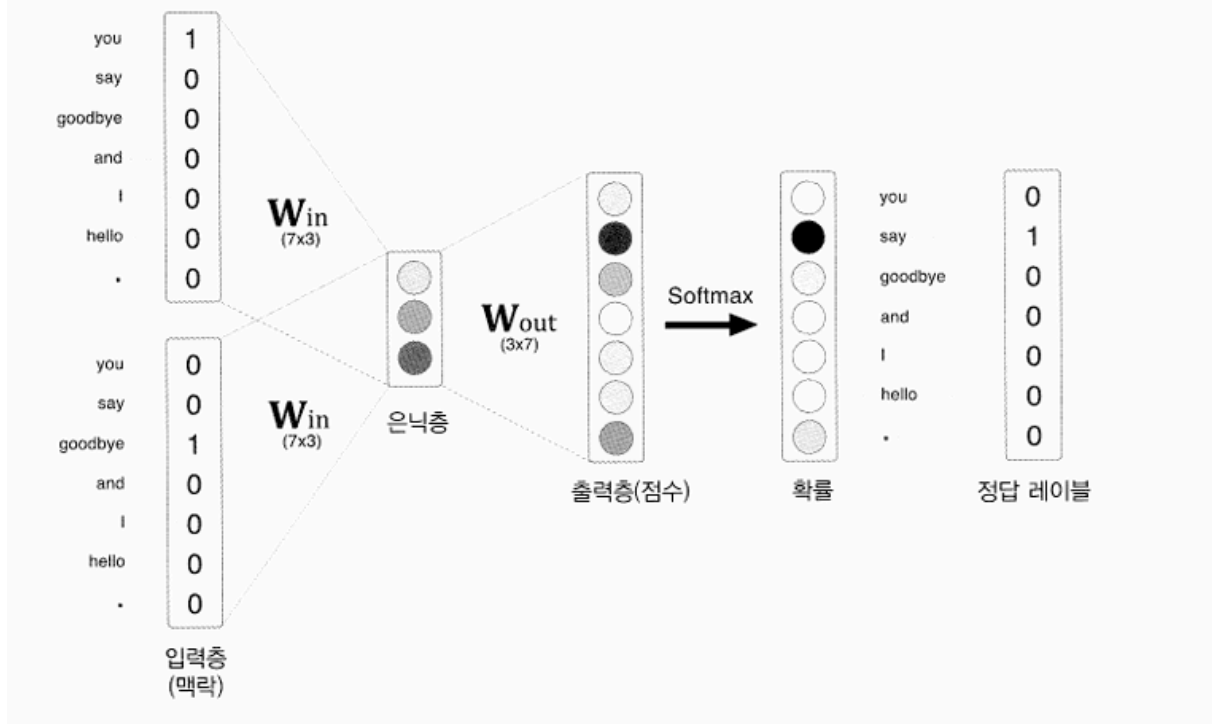


2개의 MatMul 계층이 있고, 더해진 값에 0.5를 곱해서 "평균"을 만들어 줌.

마지막에 MatMul 계층이 적용되며 비로서 "점수"가 출력되게 된다.

그리고 여기서 softmax를 적용하면 확률이 나오는 것이다.

그림 3-12 CBOW 모델의 구체적인 예(노드 값의 크기를 흑백의 진하기로 나타냄)



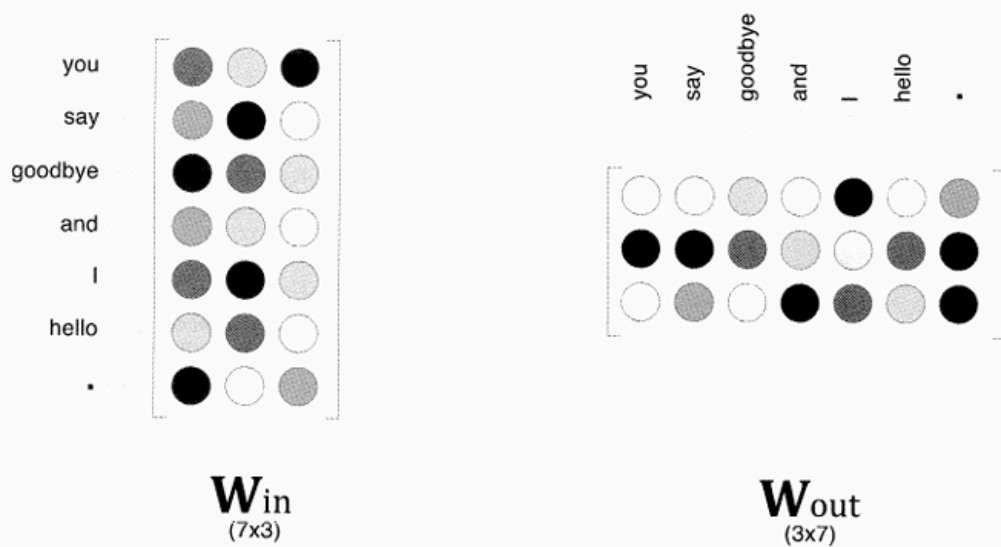
→ Loss 까지 추가한 ver

3.2.3. word2vec의 가중치와 분산 표현

word2vec에 사용되는 신경망은 2개가 있는데

1. 입력 측 완전연결계층의 가중치 → 각 단어의 분산 표현
2. 출력 측 완전연결계층의 가중치 → 단어의 의미가 인코딩된 벡터가 저장

그림 3-15 각 단어의 분산 표현은 입력 측과 출력 측 모두의 가중치에서 확인할 수 있다.



word2vec에서 주로 사용하는 최종적 단어의 분산 표현은 **입력 측의 가중치**만" 이
용하는 것

3.3. 학습데이터 준비

그림 3-16 말뭉치에서 맥락과 타깃을 만드는 예

말뭉치	맥락(contexts)	타깃
you <u>say</u> goodbye and I say hello .	you, goodbye	say
you say <u>goodbye</u> and I say hello .	say, and	goodbye
you say goodbye <u>and</u> I say hello .	goodbye, I	and
you say goodbye and <u>I</u> say hello .	and, say	I
you say goodbye and I <u>say</u> hello .	I, hello	say
you say goodbye and I say <u>hello</u> .	say, .	hello

```
text = 'You say goodbye and I say hello'
corpus, word_to_id, id_to_word = preprocess(text)
print(corpus)
```

```
[0, 1, 2, 3, 4, 1, 5]
```

```
print(id_to_word)
```

```
{0: 'you', 1: 'say', 2: 'goodbye', 3: 'and', 4: 'i', 5: 'hello'}
```

제안된 코드에 라이선스가 적용될 수 있습니다. | KingXHJ/X-Readings

```
def create_contexts_target(corpus, window_size=1):
    target = corpus[window_size:-window_size] # 중심 단어만 추림
    contexts = [] # 주변 단어 저장용

    for idx in range(window_size, len(corpus)-window_size):
        cs = [] # 하나의 중심 단어 주변 context들

        for t in range(-window_size, window_size + 1):
            if t == 0:
                continue # 중심 단어는 건너뛴
            cs.append(corpus[idx + t]) # 주변 단어 저장

        contexts.append(cs) # 전체 contexts에 추가

    return np.array(contexts), np.array(target)
```

```
contexts, target = create_contexts_target(corpus, window_size=1)
```

```
print(contexts)
```

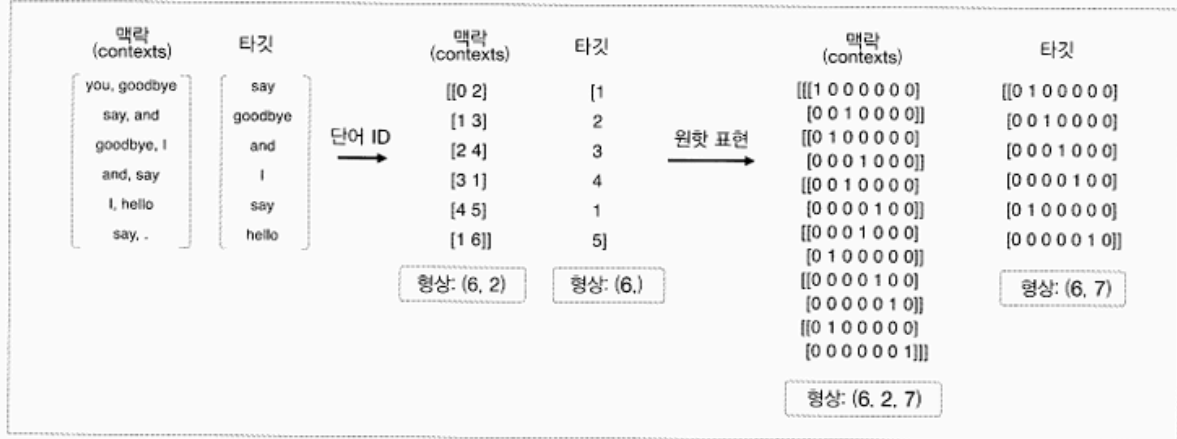
```
[[0 2]
 [1 3]
 [2 4]
 [3 1]
 [4 5]]
```

```
print(target)
```

```
[1 2 3 4 1]
```

3.3.2. 원-핫 표현으로 변환

그림 3-18 '맥락'과 '타겟'을 원핫 표현으로 변환하는 예



```
test = 'You say goodbye and I say hello.'
```

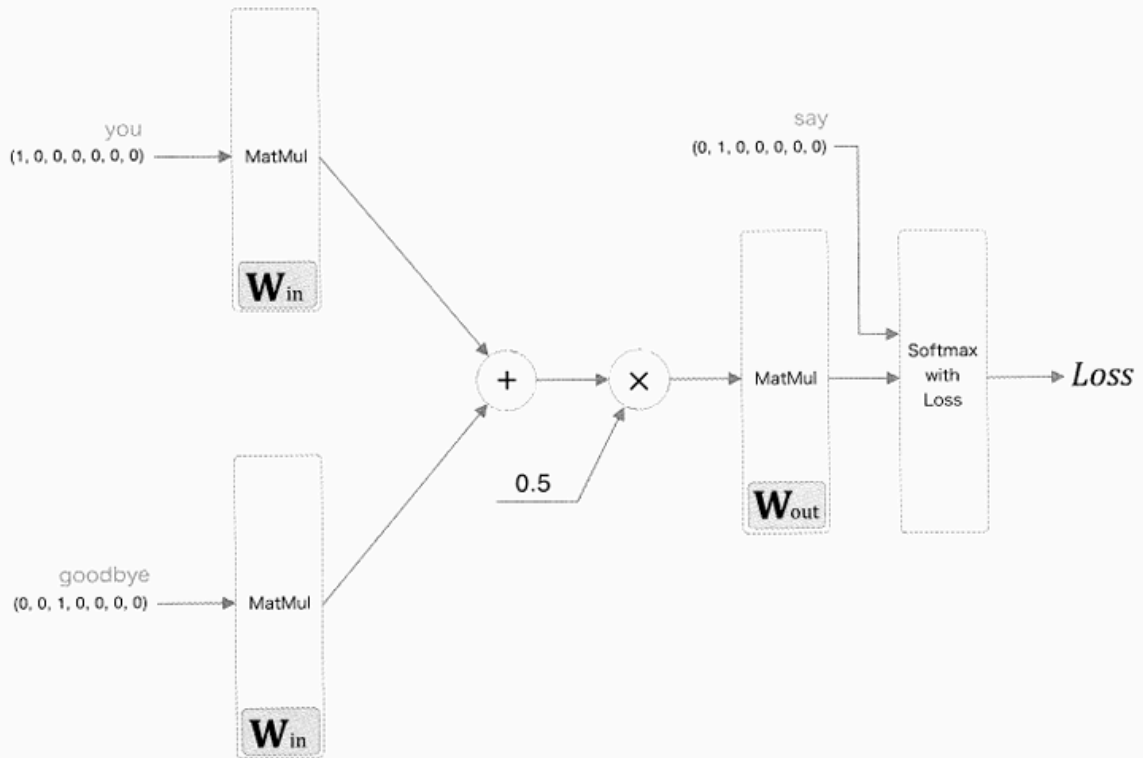
```
corpus, word_to_id, id_to_word = preprocess(test)
```

```
contexts, target = create_contexts_target(corpus, window_size=1)
```

```
vocab_size = len(word_to_id)
target = convert_one_hot(target, vocab_size)
contexts = convert_one_hot(contexts, vocab_size)
```

3.4. CBOW 모델 구현

그림 3-19 CBOW 모델의 신경망 구성



```

class SimpleCBOW:
    def __init__(self, vocab_size, hidden_size):
        V,H = vocab_size, hidden_size # 하이퍼파라미터 받아오기
        #V가 전체 단어 갯수(어휘 수)
        #H가 은닉층의 뉴런 수 ( 임베딩 벡터 차원 수 )

        W_in = 0.01 * np.random.randn(V,H).astype('f')
        W_out = 0.01 * np.random.randn(H,V).astype('f')

        self.in_layer0 = MatMul(W_in)
        self.in_layer1 = MatMul(W_in)
        self.out_layer = MatMul(W_out)
        self.loss_layer = SoftmaxWithLoss()

        layers = [self.in_layer0, self.in_layer1, self.out_layer]
        self.params, self.grads = [],[]

        for layer in layers:
            self.params = self.params + layer.params
            self.grads = self.grads + layer.grads

        self.word_vecs = W_in

```

```

def forward(self, contexts, target):
    h0 = self.in_layer0.forward(contexts[:,0])
    h1 = self.in_layer1.forward(contexts[:,1])
    h = (h0 + h1) * 0.5
    score = self.out_layer.forward(h)
    loss = self.loss_layer.forward(score, target)
    return

```

```

def backward(self, dout=1):
    ds = self.loss_layer.backward(dout)
    da = self.out_layer.backward(ds)
    da = da*0.5

    self.in_layer1.backward(da)
    self.in_layer0.backward(da)

    return None

```


3.4.1. 학습 코드 구현

```
from common.trainer import Trainer
from common.optimizer import Adam
from simple_cbow import SimpleCBOW
from common.util import preprocess, create_contexts_target, convert_one_hot
```

```
window_size = 1
hidden_size = 5
batch_size = 3
max_epoch = 1000
```

```
text = "You say goodbye and I say hello"
corpus, word_to_id, id_to_word = preprocess(text)
```

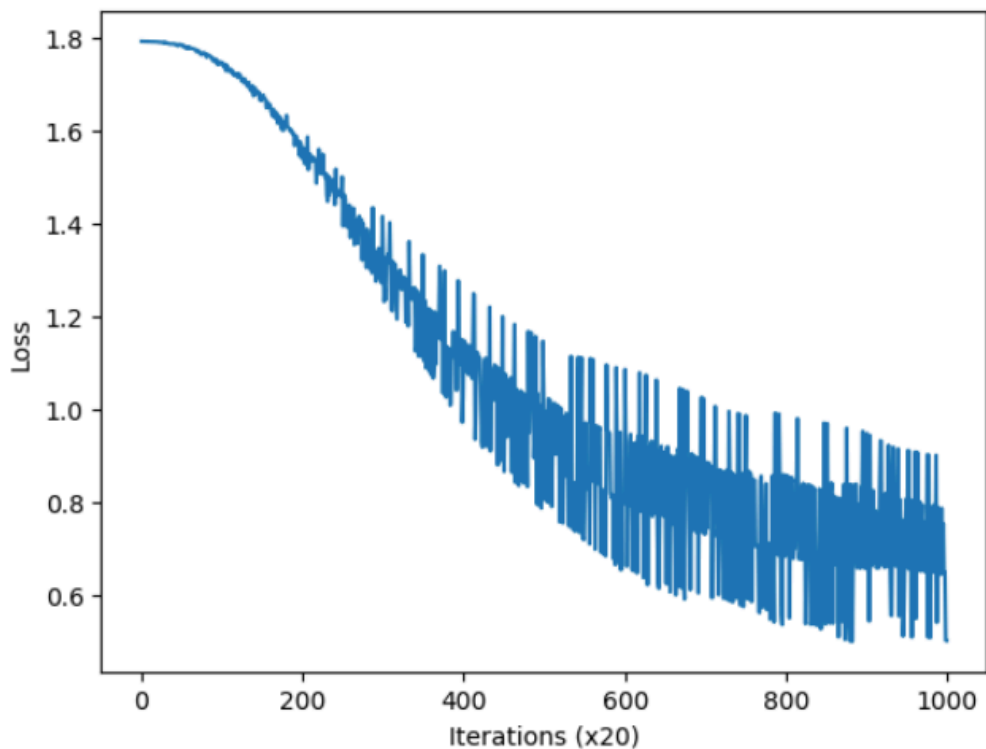
```
vocab_size = len(word_to_id)
contexts, target = create_contexts_target(corpus, window_size)
target = convert_one_hot(target, vocab_size)
contexts = convert_one_hot(contexts, vocab_size)
```

```
model = SimpleCBOW(vocab_size, hidden_size)
optimizer = Adam()
trainer = Trainer(model, optimizer)
```

```
trainer.fit(contexts, target, max_epoch, batch_size)
trainer.plot()
```

```
trainer.fit(contexts, target, max_epoch, batch_size)
trainer.plot()
```

	에폭	969		반복	1 / 1		시간	0[s]		손실	0.76
	에폭	970		반복	1 / 1		시간	0[s]		손실	0.79
	에폭	971		반복	1 / 1		시간	0[s]		손실	0.66
	에폭	972		반복	1 / 1		시간	0[s]		손실	0.76
	에폭	973		반복	1 / 1		시간	0[s]		손실	0.65
	에폭	974		반복	1 / 1		시간	0[s]		손실	0.65
	에폭	975		반복	1 / 1		시간	0[s]		손실	0.65
	에폭	976		반복	1 / 1		시간	0[s]		손실	0.90
	에폭	977		반복	1 / 1		시간	0[s]		손실	0.51
	에폭	978		반복	1 / 1		시간	0[s]		손실	0.90
	에폭	979		반복	1 / 1		시간	0[s]		손실	0.51
	에폭	980		반복	1 / 1		시간	0[s]		손실	0.51
	에폭	981		반복	1 / 1		시간	0[s]		손실	0.79
	에폭	982		반복	1 / 1		시간	0[s]		손실	0.65
	에폭	983		반복	1 / 1		시간	0[s]		손실	0.65
	에폭	984		반복	1 / 1		시간	0[s]		손실	0.76
	에폭	985		반복	1 / 1		시간	0[s]		손실	0.79
	에폭	986		반복	1 / 1		시간	0[s]		손실	0.65
	에폭	987		반복	1 / 1		시간	0[s]		손실	0.90
	에폭	988		반복	1 / 1		시간	0[s]		손실	0.54
	에폭	989		반복	1 / 1		시간	0[s]		손실	0.79
	에폭	990		반복	1 / 1		시간	0[s]		손실	0.65
	에폭	991		반복	1 / 1		시간	0[s]		손실	0.64
	에폭	992		반복	1 / 1		시간	0[s]		손실	0.76
	에폭	993		반복	1 / 1		시간	0[s]		손실	0.64
	에폭	994		반복	1 / 1		시간	0[s]		손실	0.79
	에폭	995		반복	1 / 1		시간	0[s]		손실	0.65
	에폭	996		반복	1 / 1		시간	0[s]		손실	0.75
	에폭	997		반복	1 / 1		시간	0[s]		손실	0.64
	에폭	998		반복	1 / 1		시간	0[s]		손실	0.65
	에폭	999		반복	1 / 1		시간	0[s]		손실	0.50
	에폭	1000		반복	1 / 1		시간	0[s]		손실	0.50



```
word_vecs = model.word_vecs
for word_id, word in id_to_word.items():
    print(word, word_vecs[word_id])
```

```
you [-0.9207357  0.8711709 -0.8968376 -0.8307736  0.8480286]
say [-0.7205258  0.7447087 -0.6891007  0.98623335 -0.9657756 ]
goodbye [-0.39273375  0.47851625 -0.51913726 -0.87600595  0.8682295 ]
and [-0.7313212  0.7595073 -0.6948302  0.96969885 -0.98520905]
i [-0.39063495  0.4591327 -0.4847614 -0.8697367  0.89690036]
hello [-0.8748234  0.8718964 -0.86393255 -0.79936075  0.82263654]
```

3.5. word2vec 보충

CBOW 모델을 “확률” 관점에서 본다면...

3.5.1. CBOW 모델과 확률

우선 확률은 $P()$ 라고 쓸 예정

→ 그래서 $P(A)$ 라면 A라는 현상이 일어날 확률이 된다.

동시에 일어날 확률은 $P(A,B)$ 가 되는 것이고 이 뜻은

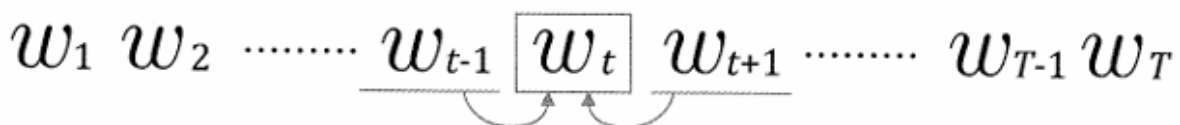
→ A와 B가 동시에 일어날 확률이 된다.

사후 확률 :

$P(A|B)$ 이런식으로 쓰는데, “사건이 일어난 후의 확률”이라는 뜻이다.

→ B가 주어졌을때 (B라는 정보) A가 일어날 확률인 것이다.

그림 3-22 word2vec의 CBOW 모델(맥락의 단어로부터 타깃 단어를 추측)



여기서 t번째 단어에 대해 윈도우 크기가 1인 맥락으로 생각하면

$W_{\{t-1\}}$ 와 $W_{\{t+1\}}$ 이 주어졌을 때 타겟이 $W_{\{t\}}$ 가 될 확률을 수식으로 쓴다면
 $\Rightarrow P (W_{\{t\}} \mid W_{\{t-1\}}, W_{\{t+1\}})$

$P (W_{\{t\}} \mid W_{\{t-1\}}, W_{\{t+1\}})$ 이 식을 이용하여 CBOW 모델의 손실 함수도 표현할 수 있다.

교차 엔트로피 오차를 떠올려봤을때

$$L = - \sum_k t_k \log y_k$$

Loss 의 식은 다음과 같았고

$y_{\{k\}}$ 는 k번째에 해당하는 사건이 일어날 확률을 말했다.

$t_{\{k\}}$ 는 정답 레이블이면서 원핫 벡터로 표현이 된다.

그렇다면 여기에서는 $W_{\{t\}}$ 가 발생이니까 $W_{\{t\}}$ 에 해당하는 원소만 1이고 나머지는 0이 되는 것

$$L = -\log P(w_t \mid w_{t-1}, w_{t+1})$$

**** 참고 :** 이렇게 마이너스를 붙인 것의 이름은 **음의 로그 기능도** 라고 불린다

이렇게 나타낸 식은 샘플데이터 하나에 대한 손실 함수이고

말 문치 전체로 확장하게되면

$$L = -\frac{1}{T} \sum_{t=1}^T \log P(w_t \mid w_{t-1}, w_{t+1})$$

다음과 같은 식이 된다.

따라서 이 손실 함수의 값을 가능한 작게 만드는 것이 CBOW 모델 학습의 목표
이때 가중치 매개변수가 우리가 얻고자 하는 단어의 분산 표현이다.

(윈도우가 1일 경우엔 다음과 같지만, 다른 크기일 경우에도 가능하다.)

3.5.2. skip-gram 모델

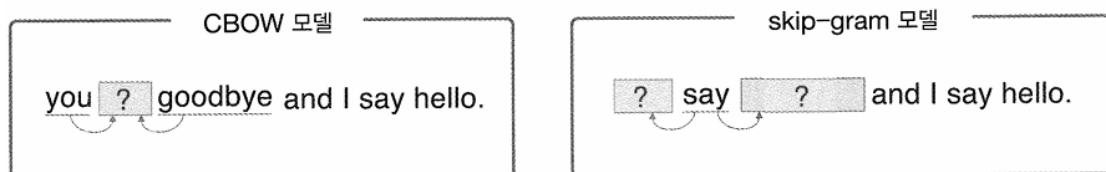
word2vec이 제안하는 모델은 2개인데

1. CBOW 모델
2. skip - gram

지금까지는 CBOW 모델을 봤고, skip-gram을 보겠다

skip-gram은 CBOW에서 다루는 맥락과 타겟을 역전시킨 것이다

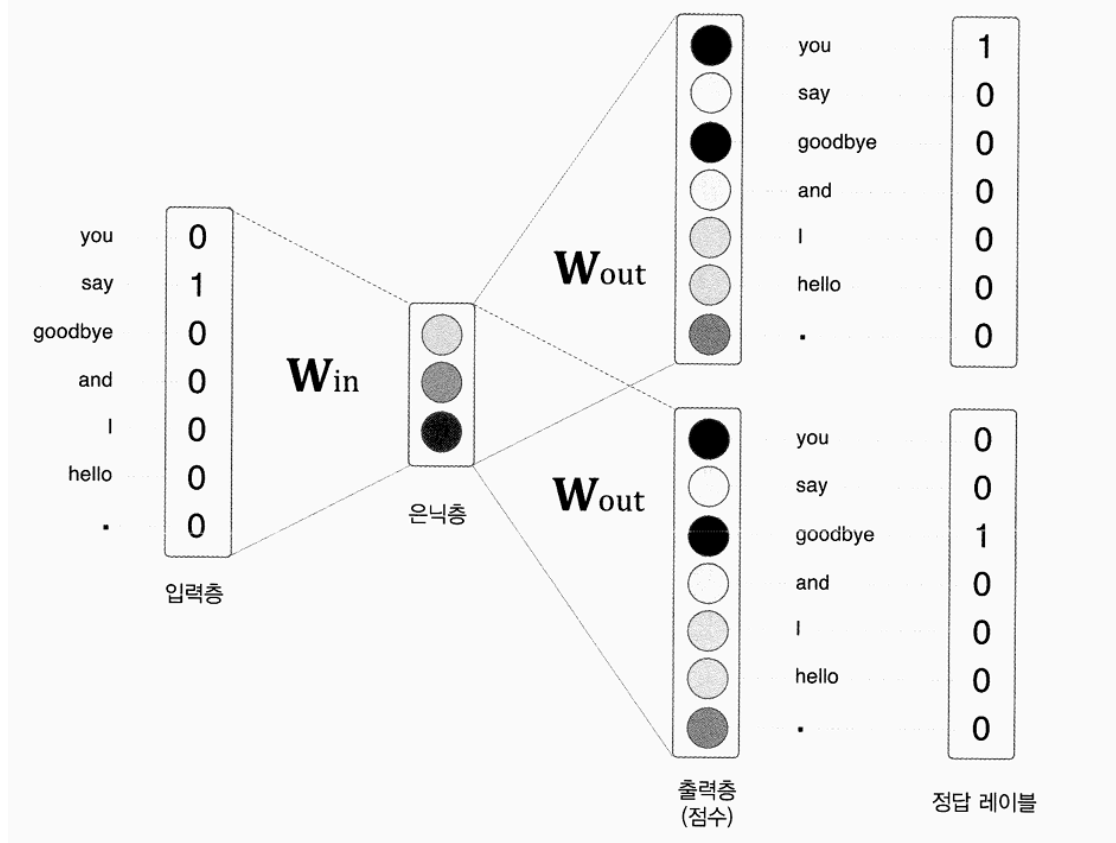
그림 3-23 CBOW 모델과 skip-gram 모델이 다루는 문제



CBOW모델은 여러 맥락이 있고, 그 맥락으로부터 중앙 단어 (target)를 추측한다

skip-gram 모델은 중앙 단어(target)로부터 주변의 여러 단어(맥락)을 추측한다

그림 3-24 skip-gram 모델의 신경망 구성 예



보다시피 입력층은 하나다

하지만 출력층은 맥락의 수 만큼 존재한다

그래서 각 출력층에서는 softmax with Loss 계층으로 개별적으로 손실을 구한다.
이 손실을 모두 더한 값을 최종 손실로 한다.

skip-gram 모델을 확률 표기로 나타내면

$$P(W_{t-1}, W_{t+1} | W_t)$$

⇒ W_t 가 주어졌을때, W_{t-1} 와 W_{t+1} 이 동시에 일어날 확률

조건부 독립이라고 가정하면

$$P(w_{t-1}, w_{t+1} | w_t) = P(w_{t-1} | w_t)P(w_{t+1} | w_t)$$

교차 엔트로피 오차에 적용해서 skip-gram모델의 손실 함수 유도

$$\begin{aligned} L &= -\log P(w_{t-1}, w_{t+1} | w_t) \\ &= -\log P(w_{t-1} | w_t) P(w_{t+1} | w_t) \\ &= -(\log P(w_{t-1} | w_t) + \log P(w_{t+1} | w_t)) \end{aligned}$$

이 유도 과정에서는 $\log xy = \log x + \log y$ 라는 로그의 성질을 이용

skip-gram 손실 함수

$$L = -\frac{1}{T} \sum_{t=1}^T (\log P(w_{t-1} | w_t) + \log P(w_{t+1} | w_t))$$

skip-gram 모델은 맥락의 수 만큼 추측

so, 손실 함수는 각 맥락에서 구한 손실의 총합이어야 한다.

↔ CBOW 모델은 타겟 하나의 손실을 구한다

참고로 둘중에

단어 분산 표현의 정밀도 측면에서 skip-gram모델이 더 좋기 때문에 skip-gram을 사용해야한다.

특히, 말뭉치가 커질수록 저빈도 단어나 유추 문제의 성능 면에서 뛰어난 성능을 보인다.

BUT, 학습 속도 면에서는 CBOW모델이 빠르다

Why? skip-gram은 손실을 맥락의 수만큼 구해야해서 계산 비용이 커지기 때문이다.

3.5.3. 통계 기반 vs 추론 기반

지금까지 통계 기반 기법 & 추론 기반 기법 (word2vec)을 봤는데

→ 학습하는 틀 측면

통계 기반 :

말뭉치의 전체 통계로부터 1회 학습하여 단어의 분산 표현 얻기

추론 기반 :

말뭉치를 일부분씩 여러 번 보며 학습 (미니배치)

1. 어휘에 새로운 단어가 생겨서 분산 표현을 갱신해야한다?

통계 기반 :

계산을 처음부터 다시 한다

동시발생 행렬을 다시 만들고 SVD를 수행하는 일련의 작업을 다시 해야함

추론 기반 :

매개변수를 다시 학습 가능

지금까지 학습한 가중치를 초깃값으로 다시 삼아서 학습

⇒ 기존 학습 경험을 해치지 않으면서 단어의 분산 표현을 효율적으로 갱신할 수 있음

👑추론 기반 우세👑

2. 단어의 분산 표현 성격이나 정밀도 면에선?

통계 기반 :

단어의 유사성이 인코딩된다

추론 기반 :

단어의 유사성 + 단어 사이의 패턴까지 파악되어 인코딩된다

유추 문제를 풀 수 있다는 이야기로 유명한 것을 방증한다.

ex) king - man + woman $\Rightarrow$ queen

우열을 가릴 수 없음

두가지의 기법은 서로 관련이 있다 (통계 기반 & 추론 기반)

모두 말뭉치 전체의 동시발생 행렬에 특수한 행렬 분해를 적용한 것

추가적으로 word2vec 이후

추론 기반 기법 + 통계 기반 기법 ==융합==> GloVe 기법 등장

Glove의 아이디어

: 말뭉치 전체의 통계 정보를 손실 함수에 도입

$\Rightarrow$ and 미니 배치로 학습!

3.6. 정리

이번 장에서 배운 내용

- 추론 기반 기법은 추측하는 것이 목적이며, 그 부산물로 단어의 분산 표현을 얻을 수 있다.
- word2vec은 추론 기반 기법이며, 단순한 2층 신경망이다.
- word2vec은 skip-gram 모델과 CBOW 모델을 제공한다.
- CBOW 모델은 여러 단어(맥락)로부터 하나의 단어(타겟)를 추측한다.
- 반대로 skip-gram 모델은 하나의 단어(타겟)로부터 다수의 단어(맥락)를 추측한다.
- word2vec은 가중치를 다시 학습할 수 있으므로, 단어의 분산 표현 갱신이나 새로운 단어 추가를 효율적으로 수행할 수 있다.